cam_async_fifo.v, explained part by part

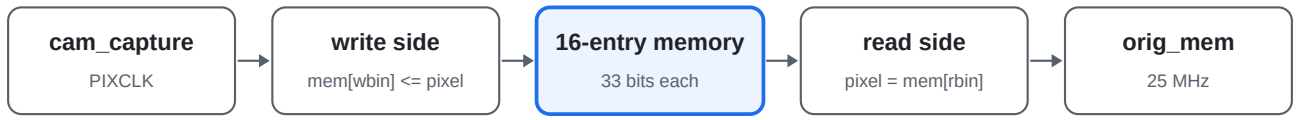
Asynchronous FIFO that moves camera pixels safely from the camera clock to the 25 MHz system clock. DE2-115 live JPEG compression project. Prepared by Dev Vijay for the project review.

1. What the module does and why it is needed

cam_capture produces pixels on the camera's clock (**PIXCLK**). The memory, compression engine and VGA run on our **25 MHz** clock. The two clocks are not related: their edges drift past each other. If a multi-bit value is read by the other clock while it is changing, some bits may be old and some new, giving a wrong value. This is the **clock domain crossing** problem.

A **FIFO** (first in, first out) solves it: the camera side writes pixels into a small memory, the 25 MHz side reads them out in the same order. Only the read and write **positions** cross between clocks, and they cross in **Gray code** so that a value caught mid-change is never badly wrong.

How a pixel crosses



pointers cross as Gray code through 2 flip-flops in each direction

Why 16 entries are enough

The camera produces a pixel only on odd rows and odd columns, 1 per 4 camera clocks on average (2 per 4 during odd rows). The 25 MHz side removes one pixel every clock (ren is always 1). The FIFO therefore never holds more than a few entries; 16 gives a large safety margin.

2. The code, part by part

Part A - Parameters and ports (lines 11-24)

Lines 11-24

```
module cam_async_fifo #(
    parameter W  = 33,                // data width
    parameter AW = 4                  // 2^AW = 16 entries
)(
    input  wire          wclk,
    input  wire          wen,
    input  wire [W-1:0]  wdata,
    output wire          wfull,

    input  wire          rclk,
    input  wire          ren,
    output wire [W-1:0]  rdata,       // valid whenever rempty = 0
    output wire          rempty
);
```

- **Parameters** make the FIFO reusable: W = data width (33 bits = 17-bit address + 16-bit pixel), AW = address width (4, so 2^4 = 16 entries). The top module sets them with #(.W(33), .AW(4)).
- Write side: **wclk, wen, wdata, wfull**. Read side: **rclk, ren, rdata, rempty**.
- This is a **first-word fall-through** FIFO: rdata already shows the oldest entry whenever rempty = 0; ren just moves on to the next one.

Part B - Storage and pointers (lines 26-30)

Lines 26-30

```
    reg [W-1:0] mem [0:(1<<AW)-1];

    reg [AW:0] wbin = 0, wgray = 0, rbin = 0, rgray = 0;
    reg [AW:0] rgray_w1 = 0, rgray_w2 = 0;     // read pointer seen by the write side
    reg [AW:0] wgray_r1 = 0, wgray_r2 = 0;     // write pointer seen by the read side
```

- **mem** is 16 entries of 33 bits.
- **wbin / rbin** are the write and read positions in normal binary; **wgray / rgray** are the same values in Gray code.
- The pointers have **AW + 1 = 5 bits** although only 4 are needed to address 16 entries. The extra top bit counts how many times the pointer has wrapped round, which is how full and empty are told apart (Part E).
- **rgray_w1, rgray_w2**: the read pointer passed through 2 flip-flops on the write clock. **wgray_r1, wgray_r2**: the write pointer passed through 2 flip-flops on the read clock.

What Gray code is

In Gray code only **one bit changes** between consecutive numbers. Binary 3 -> 4 changes three bits (011 -> 100); if caught halfway it could read as any of 8 values. In Gray code 3 -> 4 is 010 -> 110: only one bit changes, so a value caught mid-change is either the old or the new number, both safe.

Number	0	1	2	3	4	5	6	7
Binary	000	001	010	011	100	101	110	111
Gray = bin ^ (bin >> 1)	000	001	011	010	110	111	101	100

Part C - Write side, camera clock (lines 32-44)

Lines 32-44

```
    // ---------- write side (camera clock) ----------
    wire [AW:0] wbin_n  = wbin + {{AW{1'b0}}, (wen & ~wfull)};
    wire [AW:0] wgray_n = (wbin_n >> 1) ^ wbin_n;

    always @(posedge wclk) begin
        if (wen & ~wfull) mem[wbin[AW-1:0]] <= wdata;
        wbin     <= wbin_n;
        wgray    <= wgray_n;
        rgray_w1 <= rgray;
        rgray_w2 <= rgray_w1;
    end

    assign wfull = (wgray == {~rgray_w2[AW:AW-1], rgray_w2[AW-2:0]});
```

- **wbin_n** is the next write position: +1 when writing and not full.
- **wgray_n = (wbin_n >> 1) ^ wbin_n** converts binary to Gray (shift right by one, then XOR).
- On each camera clock: store the pixel at mem[wbin], update both pointers, and pass the read pointer through the 2-flip-flop synchroniser (rgray_w1 then rgray_w2).
- **{{AW{1'b0}}, (wen & ~wfull)}** is a 5-bit number equal to 0 or 1; the replication {AW{1'b0}} makes 4 zero bits.

Part D - Read side, 25 MHz clock (lines 46-58)

Lines 46-58

```
    // ---------- read side (25 MHz) ----------
    wire [AW:0] rbin_n  = rbin + {{AW{1'b0}}, (ren & ~rempty)};
    wire [AW:0] rgray_n = (rbin_n >> 1) ^ rbin_n;

    always @(posedge rclk) begin
        rbin     <= rbin_n;
        rgray    <= rgray_n;
        wgray_r1 <= wgray;
        wgray_r2 <= wgray_r1;
    end

    assign rempty = (rgray == wgray_r2);
    assign rdata  = mem[rbin[AW-1:0]];
```

- The same structure, mirrored: **rbin** advances when reading and not empty; the write pointer is passed through wgray_r1, wgray_r2 into the read clock.
- **rdata = mem[rbin]** is read directly (no clock), which is why the oldest value is always visible.

Part E - Full and empty flags

- **Empty** (line 57): the read pointer has caught up with the write pointer, so rgray == the synchronised wgray.
- **Full** (line 44): the write pointer is exactly 16 ahead of the read pointer. In binary that means the same lower 4 bits and a different top bit. In Gray code this shows up as the **top two bits inverted** and the rest equal, which is exactly **{~rgray_w2[AW:AW-1], rgray_w2[AW-2:0]}**.
- Because each side sees the other's pointer 2 clocks late, it can only think the FIFO is fuller (write side) or emptier (read side) than it really is. That is the safe direction: data is never overwritten or read twice.

3. Questions the professor may ask

Q: Why not just connect the camera data to the memory directly?

The memory runs on 25 MHz and the camera on PIXCLK. Sampling multi-bit data from another clock can catch it mid-change and store wrong values. The FIFO crosses safely.

Q: What is metastability?

If a signal changes exactly at a flip-flop's clock edge, the flip-flop can stay between 0 and 1 for a short time. A second flip-flop gives it a full clock period to settle; that is the 2-flip-flop synchroniser.

Q: Why Gray code for the pointers?

Only one bit changes per step, so even if the pointer is caught while changing, the value seen is either the old or the new position, never a random one.

Q: Why is the pointer one bit wider than needed?

With 4 bits, 'read = write' would mean both empty and full. The extra wrap-around bit separates the two cases.

Q: Can the FIFO overflow?

Not in this design: the camera writes on average 1 pixel per 4 of its clocks, the read side removes one every clock, and wfull would stop writes anyway.

Q: Why did you write it yourself instead of using an Intel FIFO IP?

The project requires Verilog only, and this standard design is about 40 lines.